

Listado de tareas

Matemáticas del Aprendizaje de Máquina

Autor:
David Felipe Angel Herrera

27 de julio de 2026

1. La desigualdad de Hoeffding no se aplica

Ejercicio 1.10

Aquí hay un experimento que ilustra la diferencia entre una sola urna y múltiples urnas. Realice una simulación por computadora para lanzar 1000 monedas justas. Lance cada moneda de forma independiente 10 veces. Centrémonos en 3 monedas de la siguiente manera: c_1 es la primera moneda lanzada; C_{rand} es una moneda que usted elige al azar; C_{min} es la moneda que tuvo la frecuencia mínima de caras (escoja la anterior en caso de empate). Sean ν_1 , ν_{rand} y ν_{min} la fracción de caras que obtiene para las tres monedas respectivas.

- ¿Cuál es μ para las tres monedas seleccionadas?
- Repita todo este experimento un gran número de veces (por ejemplo, 100.000 ejecuciones de todo el experimento) para obtener varias instancias de ν_1 , ν_{rand} y ν_{min} y grafique los histogramas de las distribuciones de ν_1 , ν_{rand} y ν_{min} . Observe que qué monedas terminan siendo C_{rand} y C_{min} puede diferir de una ejecución a otra.
- Usando (b), grafique las estimaciones para $P[|\nu - \mu| > \epsilon]$ en función de ϵ , junto con la cota de Hoeffding $2e^{-2\epsilon^2 N}$ (en el mismo gráfico).
- ¿Qué monedas obedecen la cota de Hoeffding y cuáles no? Explique por qué.
- Relacione la parte (d) con las múltiples urnas en la Figura.

Solución:

- Dado que las 1000 monedas son justas cada una de las tres seleccionadas se tendrá la probabilidad de:

$$\mu = 0,5$$

Dado que al ser eventos independientes cada lanzamiento es independiente la probabilidad de que salga una cara a la frecuencia con la cual haya salido.

- A continuación se presentan los histogramas generados por el código adjunto al presente documento:

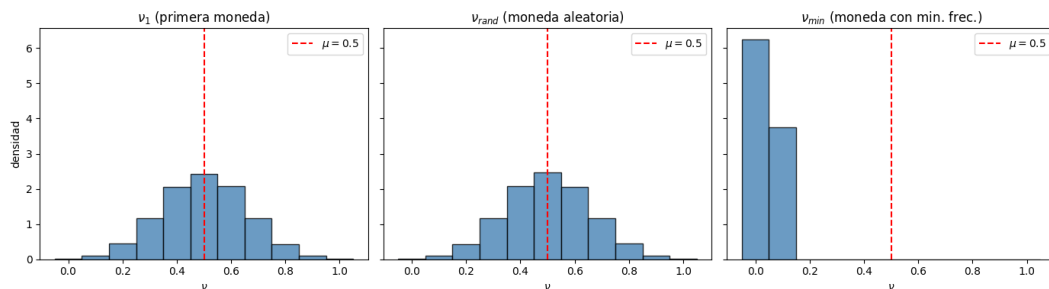


Figura 1: Histogramas sobre las monedas

Nótese que ν_1 y ν_{rand} poseen distribuciones simétricas centradas en 0.5, lo cual coincide con lo que la teoría predice, mientras que ν_{min} es una distribución sesgada hacia el 0, esto ocurre porque se elige precisamente una moneda específica que obtuviese el mínimo de caras.

- Gráfica de las estimaciones y la cota de Hoeffding:

Note que las líneas correspondientes a ν_1 y ν_{rand} coinciden casi por completo, esto a su vez coincide con la teoría.

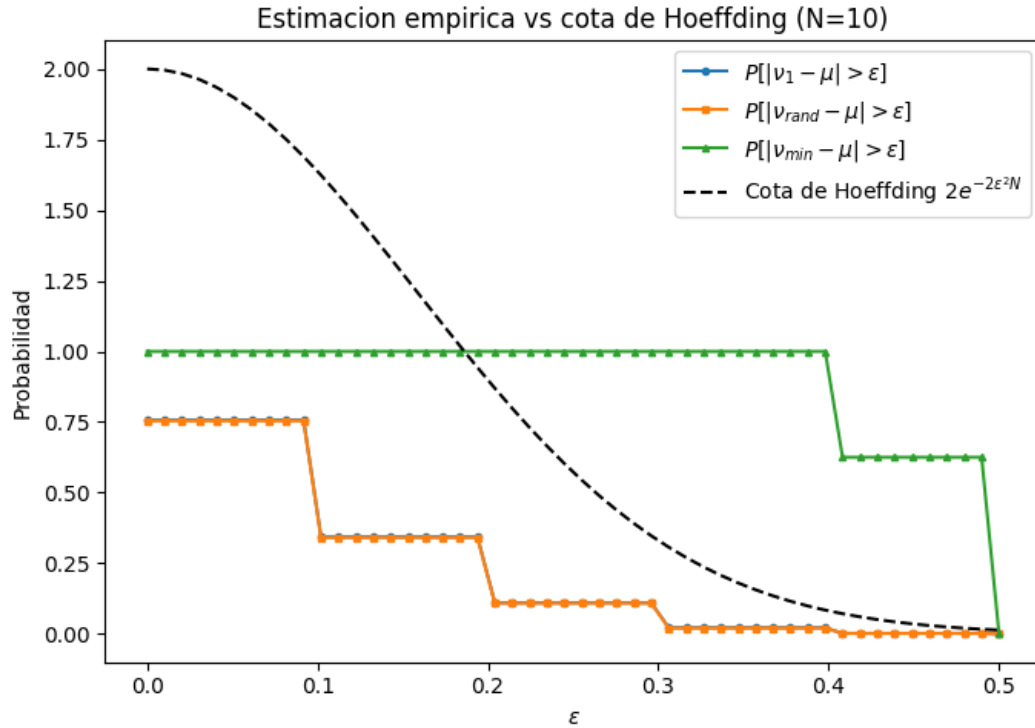


Figura 2: Estimaciones vs cota de Hoeffding

- (d) De la anterior gráfica podemos concluir con que claramente C_1 y C_{rand} satisfacen la cota de Hoeffding para todo ϵ , sin embargo la moneda C_{min} no lo hace, esto ocurre porque la selección de esta moneda ocurre después de observar los resultados, lo cual viola la hipótesis de considerar una variable aleatoria fija con anterioridad al conocimiento sobre los resultados.
- (e) La relación con la figura vista en clase (la de las urnas con bolas verdes y rojas de la desigualdad de Hoeffding) es que si seleccionamos de manera aleatoria o sin priorizar algún resultado, entonces podremos tener una buena estimación sobre la probabilidad de que se tenga un evento dado sobre una fracción de las bolas respecto a la probabilidad de que se tengan de tal modo dado. Así pues, a medida que crezca el número de muestras, podremos garantizar que la diferencia será tan pequeña como se quiera y que estará acotada por la desigualdad de Hoeffding. Sólo si se eligen de manera arbitraria desconociendo los resultados y si el tamaño muestral es lo suficientemente grande.
2. Probar que $\log_2 P(X = i)$ es la cantidad de preguntas binarias que se necesitan para representar un mensaje con esa probabilidad. Realizar la demostración de por qué se puede expresar la entropía de una variable aleatoria mediante la fórmula adjunta.

$$H(x) = - \sum_{i=1}^n P(x = i) \log_2(P(x = i))$$

Solución:

Una forma de visualizar estas preguntas binarias (digamos con respuestas del tipo sí o no) es construyendo un árbol binario codificado por un código binario con prefijos, es decir, donde cada hoja del árbol corresponde a un resultado i , y donde la profundidad de la hoja (correspondiente al número de preguntas necesarias para llegar a la

respuesta) es la longitud del código l_i .

Una desigualdad vista en el curso de teoría de la codificación acota el conjunto de longitudes $\{l_i\}$ de tal forma en que sea posible que se de una realización como un código binario prefijo, esta es la desigualdad de Kraft:

$$\sum_i 2^{-l_i} \leq 1$$

Donde es claro que el código no puede tener longitud nula, pues asumimos de la teoría que no existen códigos de longitud 0.

Si minimizamos el valor esperado del número de preguntas:

$$\begin{aligned} L &= \min_{l_i} \sum_i P(X = i) l_i \\ \text{s.a. } &\sum_i 2^{-l_i} \leq 1 \end{aligned}$$

Y empleamos multiplicadores de Lagrange para solucionar el problema de optimización, obtenemos:

$$\begin{aligned} \mathcal{L}(l_i, \lambda) &= \sum_i P(X = i) l_i + \lambda \left(\sum_i 2^{-l_i} - 1 \right) \\ \frac{\partial \mathcal{L}}{\partial l_i} &= p_i - \lambda \ln(2) 2^{-l_i} = 0 \\ &\vdots \\ 2^{-l_i} &= \frac{p_i}{\lambda \ln(2)} \end{aligned}$$

Se considera la restricción de la desigualdad de Kraft como una igualdad con el fin de considerar el caso límite en la fronte de la región a optimizar, de tal modo que aplicando lo anterior sobre la restricción:

$$\begin{aligned} \sum_i \frac{p_i}{\lambda \ln(2)} &= 1 \\ \frac{1}{\lambda \ln(2)} &= 1 \end{aligned}$$

Así pues:

$$l_i^* = -\log_2(p_i)$$

Así deducimos la entropía de Shannon a partir del conjunto de preguntas binarias codificadas por un código binario con prefijos, donde la entropía coincide con el valor esperado óptimo de preguntas binarias que determinan el resultado de la variable aleatoria X . Note que esta se maximiza (se vuelve 1) si la variable aleatoria obedece a una distribución uniforme discreta con 2^n valores posibles.

3. **Support Vector Machine:** Problema de optimización asociado a cómo encontrar los mejores pesos para un SVM y desarrollarlo con optimización convexa.

Solución:

Dado un conjunto de datos linealmente separable $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ con $y_i \in \{-1, +1\}$, buscamos el hiperplano $\mathbf{w}^\top \mathbf{x} + b = 0$ que maximiza el margen. Normalizando de forma que el punto más cercano cumpla $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$, el margen resulta $1/\|\mathbf{w}\|$, y maximizarlo equivale a resolver:

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{s.a.} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, N \end{aligned}$$

El problema es convexo, pues todas las normas vectoriales en dimensión finita son convexas al satisfacer el criterio de que sus Hessianas son siempre definidas positivas (si $\mathbf{w} \neq 0$), y las restricciones son lineales en $(\mathbf{w}, b)$, por lo que la región factible es convexa. Así pues como el problema de optimización se trata acerca de una función convexa sobre una región convexa, todo mínimo local será global. Procedemos escribiendo el Lagrangiano asociado al problema de optimización:

Lagrangiano: Introducimos multiplicadores de Lagrange $\lambda_i \geq 0$:

$$\mathcal{L}(\mathbf{w}, b, \lambda) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^N \lambda_i [y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1]$$

Donde $\lambda = (\lambda_1, \lambda_2, \dots, \lambda_N)$. Calculamos las derivadas parciales respecto a los parámetros que deseamos optimizar con el fin de resolver el problema de optimización:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{w} - \sum_i \lambda_i y_i \mathbf{x}_i &= 0 & \Rightarrow \quad \mathbf{w} &= \sum_i \lambda_i y_i \mathbf{x}_i \\ \frac{\partial \mathcal{L}}{\partial b} = - \sum_i \lambda_i y_i &= 0 & \Rightarrow \quad \sum_i \lambda_i y_i &= 0 \end{aligned}$$

Ahora se formula el problema dual con el fin de reducir la cantidad de variables a optimizar bajo las condiciones ya deducidas. Sustituyendo lo anterior de vuelta en el Lagrangiano $\mathcal{L}$:

$$\begin{aligned} \max_{\lambda} \quad & \sum_{i=1}^N \lambda_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \lambda_i \lambda_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \\ \text{s.a.} \quad & \alpha_i \geq 0 \quad \sum_{i=1}^N \alpha_i y_i = 0 \end{aligned}$$

El dual también es un problema de optimización convexa por provenir de un problema convexo, y por dualidad fuerte su óptimo coincide con el del primal. Finalmente añadimos las condiciones KKT para resolver el problema usando esta técnica cuando se tengan a disposición los datos:

$$\alpha_i [y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1] = 0 \quad \forall i$$

Esto implica que solo los puntos exactamente sobre el margen tienen $\alpha_i > 0$, son los vectores de soporte, y son los únicos que determinan $\mathbf{w}$.

4. **Perceptron:** Implementar el algoritmo del perceptron y entender por qué funciona. Hacer los ejercicios 1.2 y 1.3:

Ejercicio 1.2

Suponga que utilizamos un perceptrón para detectar mensajes de spam. Digamos que cada mensaje de correo electrónico está representado por la frecuencia de ocurrencia de palabras clave, y la salida es +1 si el mensaje se considera spam.

- (a) ¿Puede pensar en algunas palabras clave que terminarán teniendo un peso positivo grande en el perceptrón?
- (b) ¿Qué hay de las palabras clave que obtendrán un peso negativo?
- (c) ¿Qué parámetro en el perceptrón afecta directamente cuántos mensajes en la frontera terminan siendo clasificados como spam?

Ejercicio 1.3

La regla de actualización de pesos en (1.3) tiene la agradable interpretación de que se mueve en la dirección de clasificar $x(t)$ correctamente.

- (a) Muestre que $y(t)\mathbf{w}^\top(t)\mathbf{x}(t) < 0$. [Pista: $\mathbf{x}(t)$ está mal clasificado por $\mathbf{w}(t)$.]
- (b) Muestre que $y(t)\mathbf{w}^\top(t+1)\mathbf{x}(t) > y(t)\mathbf{w}^\top(t)\mathbf{x}(t)$.
- (c) En lo que respecta a clasificar $\mathbf{x}(t)$, argumente que el movimiento de $\mathbf{w}(t)$ a $\mathbf{w}(t+1)$ es un movimiento 'en la dirección correcta'.

Adicionalmente, responder estas preguntas:

- a. El algoritmo encuentra los parámetros correctos? (¿Separa los datos?)
- b. El algoritmo para en un número finito de pasos? (¿Cómo garantizar convergencia?)
- c. ¿Por qué funciona?

Solución:

Ejercicio 1.2

- (a) **Palabras con peso positivo grande:** Palabras y términos tales como “gratis”, “ganador”, “haga clic aquí”, “oferta limitada”, “suscripción”, “watch now”, “regístrese”, etc.
- (b) **Palabras con peso negativo:** Términos propios de correspondencia legítima tales como nombre del receptor, “reunión”, “importante”, “tarea”, “invitación”, “certificado”, etc.
- (c) El parámetro que controla cuántos mensajes se clasifican como spam es el umbral / bias b (o w_0), este desplaza el hiperplano de decisión sin cambiar su orientación.

Ejercicio 1.3

- (a) Como $x(t)$ está mal clasificado por $w(t)$, la predicción $\text{sgn}(w^\top(t)x(t))$ difiere de $y(t)$. Así se obtiene:

$$y(t)w^\top(t)x(t) < 0$$

- (b) Usando la regla de actualización $w(t+1) = w(t) + y(t)x(t)$:

$$\begin{aligned} y(t)w^\top(t+1)x(t) &= y(t)[w(t) + y(t)x(t)]^\top x(t) \\ &= y(t)w^\top(t)x(t) + \underbrace{y(t)^2}_{=1} \|x(t)\|^2 \\ &= y(t)w^\top(t)x(t) + \|x(t)\|^2 > y(t)w^\top(t)x(t) \end{aligned}$$

ya que $\|x(t)\|^2 > 0$ (para $x(t) \neq 0$).

- (c) $y(t)w^\top x(t)$ mide qué tan bien clasificado está $x(t)$. Por (a) este valor es negativo antes de actualizar; por (b) la actualización siempre lo incrementa en exactamente $\|x(t)\|^2$. La actualización garantiza que la sucesión de pasos sea “monótona creciente” hacia la clasificación correcta de $x(t)$.

Preguntas adicionales

- (a) Si los datos son linealmente separables, el PLA (Perceptron Learning Algorithm) siempre converge a un w que separa perfectamente los datos.
- (b) Sí, garantizado por el **teorema de Novikoff**: si existe w^* que separa los datos con margen $\rho = \min_i y_i w^{*\top} x_i / \|w^*\|$, y $R = \max_i \|x_i\|$, el número de actualizaciones está acotado por $(R/\rho)^2$.

Teorema 1 (Novikoff, 1962). *El número de actualizaciones k que realiza el PLA antes de converger satisface:*

$$k \leq \left(\frac{R}{\gamma} \right)^2$$

Donde $R = \max_i \|x_i\|$, γ es el margen de separación minimal, es decir la distancia mínima de los puntos del espacio al hiperplano, $0 < \gamma \leq y_i(w^{*\top} x_i)$ con w^* el vector director del hiperplano separador. En particular, k es **finito**, es decir, el PLA converge en un número finito de pasos.

- (c) Funciona porque cada actualización mejora estrictamente la clasificación del punto que causó el error, mientras que el teorema de Novikoff garantiza que finaliza en un número finito de pasos.

Nota: En el código se adjunta una implementación del PLA y las comprobaciones de que en efecto converge en un número finito de pasos para muestras distintas.

5. Modelar un XOR conectando varias neuronas básicas. Luego modelar XOR con perceptrones (con pesos específicos para cada uno).

Solución:

XOR **no es linealmente separable** (no existe una sola recta que separe $(0,0)$, $(1,1)$ de $(0,1)$, $(1,0)$), así que necesitamos **combinar varias neuronas básicas** en una red de 2 capas.

La idea es construir 3 perceptrones con función de activación escalón unitario:

$$\text{step}(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

Neurona	Función	w_1	w_2	bias b	Expresión
h_1	OR(x_1, x_2)	1	1	-0,5	$\text{step}(x_1 + x_2 - 0,5)$
h_2	NAND(x_1, x_2)	-1	-1	1,5	$\text{step}(-x_1 - x_2 + 1,5)$
y	AND(h_1, h_2)	1	1	-1,5	$\text{step}(h_1 + h_2 - 1,5)$

Verificación con la tabla de verdad completa:

x_1	x_2	$h_1 = \text{OR}$	$h_2 = \text{NAND}$	$y = \text{AND}(h_1, h_2)$	XOR real
0	0	0	1	0	0 ✓
0	1	1	1	1	1 ✓
1	0	1	1	1	1 ✓
1	1	1	0	0	0 ✓

Los 3 perceptrones reproducen exactamente XOR bajo una construcción clásica.

- Consultar el artículo KDD Cup 2007 Task 1 Winner Report coautorado por Miklós Kurucz y responder detalladamente cómo se logró resolver el Reto Netflix a partir de la solución del problema de factorización de matrices no negativas. (Sugerencia: Hacer un paralelo de Temas vs Temáticas y de Temáticas vs Películas, deducir cómo es el comportamiento de la factorización a partir de este paralelo).

Solución:

Contexto: Task 1 (“Who Rated What in 2006”) consiste en predecir, para 100,000 pares (usuario, película), si ese usuario calificó esa película en 2006, midiendo el desempeño con RMSE sobre la matriz binaria w_{ij} (1 si el usuario i calificó la película j , 0 en caso contrario).

Componente de factorización de matrices en valores singulares (SVD): Se construye la matriz binaria completa W (usuario $\times$ película) y se calcula su aproximación de rango bajo vía SVD:

$$W \approx U_k \Sigma_k V_k^\top$$

usando $k = 10$ dimensiones latentes. Se emplea el algoritmo de Lanczos por eficiencia en matrices dispersas de gran tamaño.

Paralelo Temas / Temáticas. La interpretación de la descomposición en valores singulares (análoga a LSA – Latent Semantic Analysis – aplicado a texto) es:

- U_k (usuario $\times k$ temas latentes): codifica cada **usuario** como una mezcla de k “temáticas” de preferencia (afinidad del usuario por cada tema latente).
- V_k (película $\times k$ temas latentes): codifica cada **película** como una mezcla de esas mismas k temáticas (cuánto pertenece la película a cada tema).
- Σ_k pondera la importancia relativa de cada temática usando valores singulares.

El producto $U_k \Sigma_k V_k^\top$ reconstruye la afinidad usuario-película **sin necesidad de que hayan interactuado directamente**: si el usuario tiene alta afinidad con la temática j y la película tiene alta pertenencia a esa misma temática, el modelo predice una probabilidad alta de calificación aunque esa combinación exacta nunca haya aparecido en los datos de entrenamiento – el mismo principio que en *topic modeling* de texto (documentos $\leftrightarrow$ temas, temas $\leftrightarrow$ palabras): El espacio latente de dimensión k actúa como un puente de baja dimensión entre dos conjuntos que de otra forma solo se relacionan a través de una matriz extremadamente dispersa.

Detalle técnico adicional: Como el muestreo de los pares (usuario, película) de Task 1 no fue uniforme sino proporcional a los márgenes (número de calificaciones por usuario/película), fue necesario ponderar la matriz antes de aplicar SVD, minimizando:

$$\sum_{ij} p_{ij} \left(w_{ij} - \sum_k \sigma_k u_{ki} v_{kj} \right)^2$$

equivalente a aplicar SVD estándar sobre $\sqrt{p_{ij}} \cdot w_{ij}$ y luego reescalar. Esta componente de SVD se combinó (por regresión lineal) con otras 4 señales del modelo (predicción de marginales, similitud ítem-ítem, y reglas de asociación) para lograr el RMSE ganador de 0,256.

7. ¿Cuál es el conjunto hipótesis del modelo de aprendizaje K Nearest Neighbours? ¿Cuál es su algoritmo de aprendizaje?

Solución:

El conjunto hipótesis depende explícitamente de la muestra de entrenamiento $S = \{(x_1, y_1), \dots, (x_N, y_N)\}$:

$$\mathcal{H} = \{g_S(x) = \text{moda}\{y_i : x_i \in N_K(x, S)\}\}$$

donde $N_K(x, S)$ son los K puntos de S más cercanos a x bajo alguna métrica. Es un modelo **no paramétrico**, es decir la “forma” de las fronteras de decisión no está limitada a priori (a diferencia de una recta o hiperplano) con $N \rightarrow \infty$ y K creciendo apropiadamente, KNN puede aproximar *cualquier* frontera de decisión, es decir, es un aproximador universal asintóticamente.

Algoritmo de aprendizaje

KNN es un método de **aprendizaje perezoso (lazy learning)**, esto es no hay una fase de optimización que ajuste parámetros. El consiste en:

- a) **Entrenamiento:** Almacenar S completo.
- b) **Predicción:** Para un punto nuevo x : Calcular la distancia de x a los N puntos de S , tomar los K más cercanos, y devolver la etiqueta mayoritaria entre ellos (clasificación) o el promedio (regresión).

Todo el “aprendizaje” ocurre en tiempo de predicción, no de entrenamiento, por eso también se le llama **instance-based learning**.

8. **Cotas de concentración:**

- a. Demostración cota Markov y 1 ejemplo.
- b. Demostración cota Chebyshev.
- c. Demostrar cota de Hoeffding (Suponiendo Lema de Hoeffding).

Solución:

Enunciado desigualdad de Markov: Si $X \geq 0$ y $a > 0$, entonces

$$P(X \geq a) \leq \frac{E[X]}{a}$$

Demostración:

$$\begin{aligned} E[X] &= \int_0^\infty x f(x) dx \\ &\geq \int_a^\infty x f(x) dx \\ &\geq \int_a^\infty a f(x) dx \\ &= a P(X \geq a) \end{aligned}$$

Despejando:

$$P(X \geq a) \leq \frac{E[X]}{a}$$

Ejemplo: Sea X el número de errores hallados en un lote, con $E[X] = 3$. Entonces

$$P(X \geq 15) \leq \frac{3}{15} = 0,2$$

La probabilidad de tener 15 o más errores es a lo más de 20 %, desconociendo la función de distribución de X .

Enunciado desigualdad de Chebyshev: Si X tiene media μ y varianza σ^2 , entonces para $k > 0$:

$$P(|X - \mu| \geq k) \leq \frac{\sigma^2}{k^2}$$

Demostración: Definimos $Y = (X - \mu)^2 \geq 0$, con $E[Y] = \sigma^2$ por ser Y el segundo momento alrededor de μ . Aplicamos Markov a Y con $a = k^2$:

$$P(Y \geq k^2) \leq \frac{E[Y]}{k^2} = \frac{\sigma^2}{k^2}$$

Pero

$$\{Y \geq k^2\} = \{(X - \mu)^2 \geq k^2\} = \{|X - \mu| \geq k\}$$

Por lo tanto:

$$P(|X - \mu| \geq k) \leq \frac{\sigma^2}{k^2}$$

Enunciado desigualdad de Hoeffding: Sean $X_1, \dots, X_N$ variables aleatorias independientes, con $X_i \in [a_i, b_i]$, $S = \sum_{i=1}^N X_i$, $\mu = E[S]$. Entonces:

$$P(|S - \mu| \geq t) \leq 2e^{\left(\frac{-2t^2}{\sum_i (b_i - a_i)^2}\right)}$$

Demostración:

Lema de Hoeffding: si X es una variable aleatoria con $a \leq X \leq b$ casi seguramente y $E[X] = \mu$, entonces para todo $\lambda \in \mathbb{R}$:

$$E[e^{\lambda X}] \leq e^{\lambda \mu} e^{\left(\frac{\lambda^2 (b-a)^2}{8}\right)}$$

Paso 1 Cota de Chernoff (Markov aplicado a $e^{\lambda X}$): para cualquier $\lambda > 0$,

$$P(S - \mu \geq t) = P(e^{\lambda(S-\mu)} \geq e^{\lambda t}) \leq \frac{E[e^{\lambda(S-\mu)}]}{e^{\lambda t}}$$

Paso 2 Independencia + Lema de Hoeffding:

$$\begin{aligned} E[e^{\lambda(S-\mu)}] &= E\left[e^{\lambda \sum_i (X_i - E[X_i])}\right] \\ &= \prod_i E\left[e^{\lambda(X_i - E[X_i])}\right] \\ &\leq \prod_i e^{\left(\frac{\lambda^2 (b_i - a_i)^2}{8}\right)} \\ &= e^{\left(\frac{\lambda^2}{8} \sum_i (b_i - a_i)^2\right)} \end{aligned}$$

Paso 3 Combinar y optimizar sobre λ :

$$P(S - \mu \geq t) \leq e^{\left(-\lambda t + \frac{\lambda^2}{8} \sum_i (b_i - a_i)^2\right)}$$

Minimizando el exponente respecto a λ :

$$\lambda^* = \frac{4t}{\sum_i (b_i - a_i)^2}$$

Sustituyendo:

$$P(S - \mu \geq t) \leq e^{\left(\frac{-2t^2}{\sum_i (b_i - a_i)^2}\right)}$$

Por simetría (aplicando el mismo argumento a $-S$), se obtiene la cota de dos colas:

$$P(|S - \mu| \geq t) \leq 2e^{\left(\frac{-2t^2}{\sum_i (b_i - a_i)^2}\right)}$$

9. Inequalities learning slide 29/30. Responder la última pregunta: μ es la causa de ν , pero podemos inferir que $\mu \approx \nu$. ¿Por qué?

Descubriendo una probabilidad

Si conocemos ν (fracción), ¿podemos decir algo sobre μ (probabilidad)?

$$\Pr[|\nu - \mu| \geq \epsilon] \leq 2e^{-2N\epsilon^2}$$

- “ $\nu = \mu$ ” es probablemente aproximadamente correcto (PAC).
- A pesar de que $\Pr$ depende de μ , la cota $2e^{-2N\epsilon^2}$ no lo hace, lo cual es bueno porque μ es desconocido.
- Para tener una menor tolerancia (ϵ) necesitamos un gran número de muestras (N).
- μ es la causa de ν , pero podemos inferir que $\mu \approx \nu$. ¿Por qué?

Solución:

Nótese que no es ν quien determina μ , sino que μ existe independientemente de que se muestree o no. La desigualdad de Hoeffding:

$$P[|\nu - \mu| \geq \epsilon] \leq 2e^{-2N\epsilon^2}$$

Nos indica que el lado derecho no depende de μ , sólo de N y ϵ . Esto significa que, sin importar cuál sea el valor desconocido de μ , la cota garantiza que ν estará cerca de μ con alta probabilidad para N lo suficientemente grande.

Es decir, aunque no se controla ni se conoce la causa (μ), la concentración del efecto (ν) alrededor de la causa permite usar el efecto observado como estimador confiable de la causa desconocida.

10. A partir de la desigualdad de Hoeffding determine, entre los parámetros ϵ y $\delta = 2Me^{-2N\epsilon^2}$, cuál controla de una mejor manera el N ? (N número de datos).

Solución:

Partiendo de

$$\delta = 2Me^{-2N\epsilon^2}$$

despejamos N :

$$N = \frac{1}{2\epsilon^2} \ln\left(\frac{2M}{\delta}\right)$$

Observamos la dependencia de cada parámetro:

- **Respecto a ϵ :**

$$N \propto \frac{1}{\epsilon^2}$$

Son inversamente proporcionales. Reducir ϵ a la mitad requiere multiplicar N por 4.

■ **Respecto a δ :**

$$N \propto \ln \left(\frac{1}{\delta} \right)$$

Depende logarítmicamente. Reducir δ implica un incremento pequeño en N .

Conclusión: ϵ controla a N de forma mucho más notoria que δ . Esto significa que exigir una mayor precisión ϵ es más costoso computacionalmente que variar el δ .

11. ¿Cuál es el número efectivo de líneas (rectas) para separar 5 puntos (en 2 etiquetas?) $\mathbb{R}^2$? (Ver todas las rectas posibles que separan 2 clases compuestas por 5 puntos, discriminar los casos en los que una recta no puede separar y así encontrar el número de rectas efectivas).
-

Solución:

Derivación combinatoria: Para maximizar el conteo, se colocan los 5 puntos formando una región convexa. Se ordenan cíclicamente según su posición en el casco convexo: p_1, p_2, p_3, p_4, p_5 .

Hecho clave: cualquier recta que separe el plano en dos mitades, al intersectar puntos en posición convexa, siempre separa el conjunto en **dos arcos contiguos** en el orden cíclico.

Se cuentan las formas de partir el ciclo de 5 puntos en dos arcos no vacíos: se eligen 2 de los 5 “huecos” entre puntos consecutivos como puntos de corte, hay $\binom{5}{2} = 10$ formas de elegir esos 2 huecos, y cada elección produce una partición en dos arcos. Como cada partición puede etiquetarse de 2 formas (cuál arco es +1 y cuál es -1):

$$2 \times \binom{5}{2} = 2 \times 10 = 20 \text{ dicotomías no triviales}$$

Sumando las 2 dicotomías triviales (todos +1, o todos -1):

$$m_{\mathcal{H}}(5) = 20 + 2 = 22$$

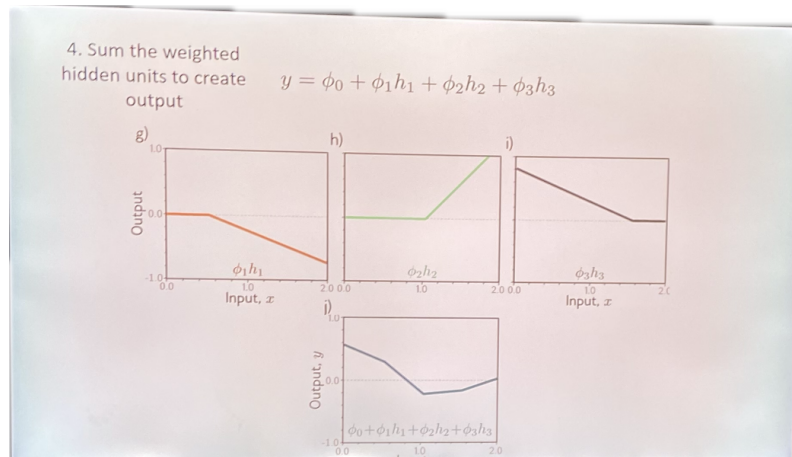
Verificación con la fórmula general de función de crecimiento del perceptrón en $\mathbb{R}^d$:

$$m_{\mathcal{H}}(N) = 2 \sum_{i=0}^d \binom{N-1}{i} = 2 \left[\binom{4}{0} + \binom{4}{1} + \binom{4}{2} \right] = 2[1 + 4 + 6] = 22$$

Casos no separables: Los $32 - 22 = 10$ etiquetados restantes son aquellos donde las etiquetas **no forman un arco contiguo** en el orden cíclico del casco convexo – patrones “alternados”, análogos al problema de XOR con 5 puntos, donde una recta tendría que zigzaguear para separar correctamente, algo que una recta no puede hacer.

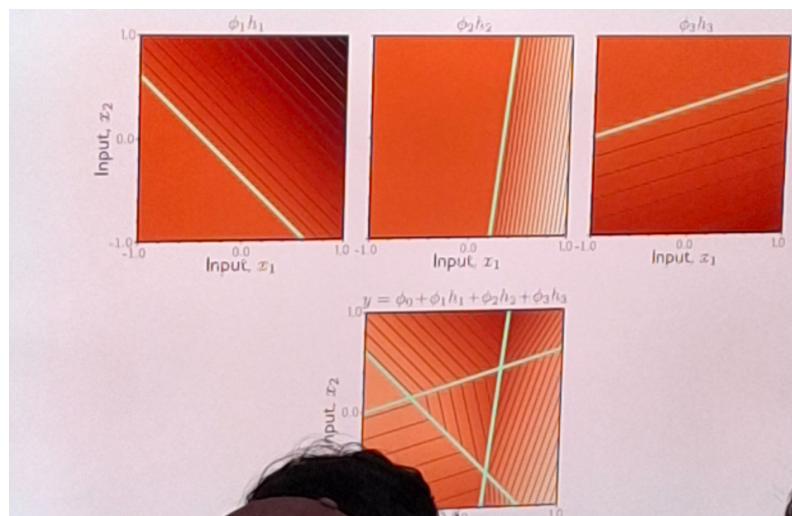
12. ¿Cuál es el origen de las junturas en una red neuronal *shallow*?

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

**Solución:**

El origen es la no linealidad que le aporta la función de activación sobre las unidades ocultas h_i , comúnmente se usa ReLU. Puesto que en la composición del paso a través de las capas es esta no linealidad la que permite modelar fronteras de decisión mucho más difíciles de lo que puede llegar a abstraer una simple regresión lineal.

13. ¿Cuántos dobleces puedo tener aquí?

**Solución:**

Según la teoría, cada unidad oculta aporta una recta de pliegue en el plano que contiene a las variables x_1, x_2 . En este caso construimos la fórmula de recurrencia para D unidades ocultas:

- Partimos notando que con 0 rectas hay una sola partición, es decir $R(0)=1$.
- Al agregar la k -ésima recta donde ya hay $k-1$ rectas trazadas (sin casos degenerados como rectas paralelas o concurrentes), por uno de los postulados de la geometría euclídea tenemos que dos rectas se cruzan en exactamente un punto, al cruzar cada una de las anteriores $k-1$ rectas se forman $k-1$ puntos de corte.
- Los $k-1$ puntos de corte parten la nueva recta en k segmentos, por uno de los postulados de separabilidad del plano de la geometría euclídea deducimos así que añadimos k nuevas regiones.

De este modo obtenemos la fórmula de recurrencia:

$$R(k) = R(k - 1) + k$$

Cuya solución es:

$$R(D) = \binom{D}{0} + \binom{D}{1} + \binom{D}{2}$$

Que en nuestro caso será para cuando se tengan 3 unidades ocultas:

$$R(3) = 1 + 3 + 3 = 7$$

Nota: Se hizo uso de IA y de LLMs para ayudarse en la comprensión de algunos temas tales como el punto 6 o la construcción de los códigos, principalmente se usó Claude Sonnet 5 con esfuerzo medio para la ayuda con los puntos, y se usó ligeramente el modelo de Gemini incorporado en Google Colab para los códigos.

Referencias:

- [1] S. J. D. Prince, Understanding Deep Learning. Cambridge, MA, USA: MIT Press, 2026.
- [2] B. Naskrecki, “Chapter 6: The Perceptron Convergence Theorem,” in Classical Foundations of Artificial Neural Networks. [Online]. Available: https://bnaskrecki.faculty.wmi.amu.edu.pl/nnets/_build/tml/part2_perceptron/ch06_convergence_theorem.html[reference:0]
- [3] M. Kurucz, A. A. Benczúr, T. Kiss, I. Nagy, A. Szabó, and B. Torma, “KDD Cup 2007 task 1 winner report,” SIGKDD Explor., vol. 9, no. 2, pp. 53–56, 2007.